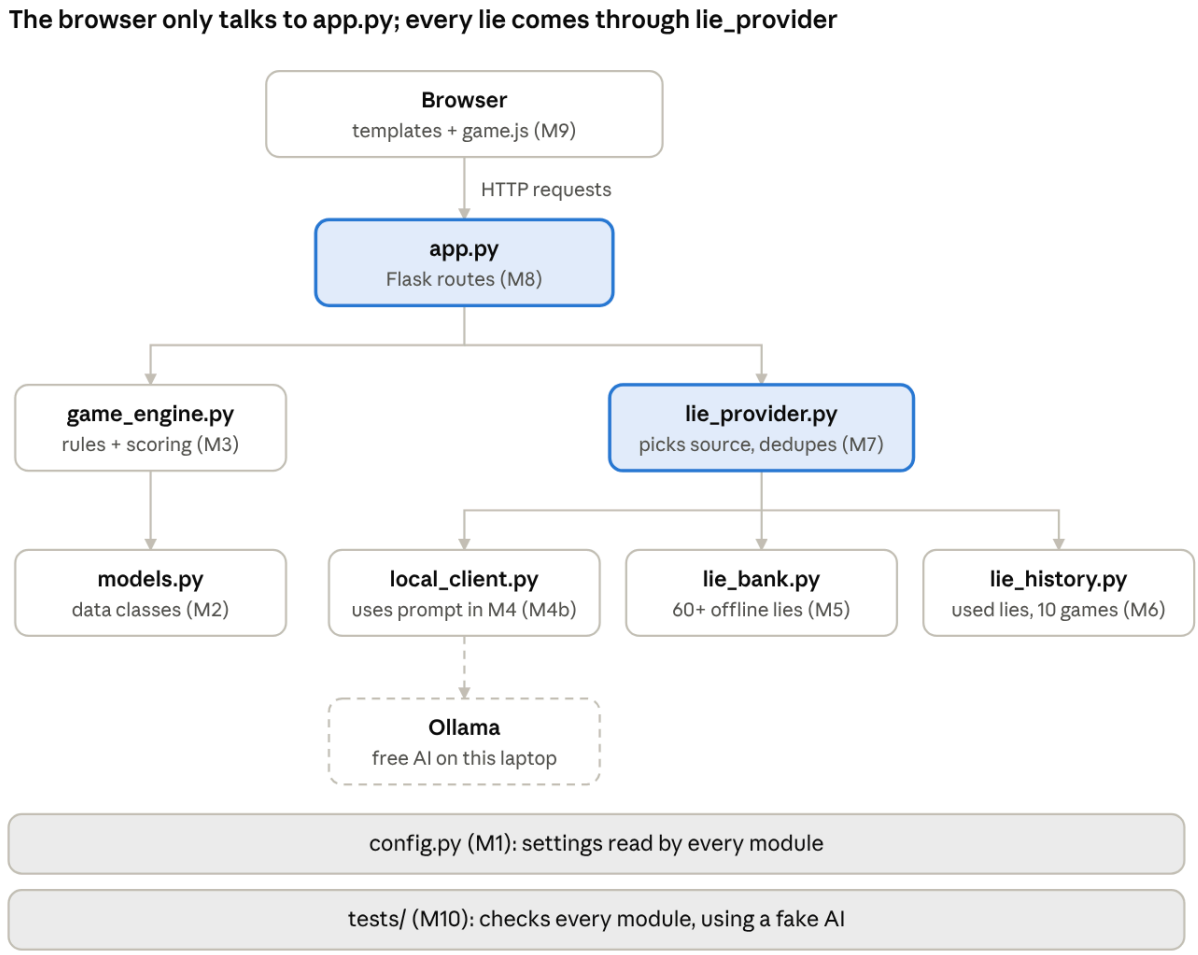
Would I Lie to You? Web App: Module Breakdown

The app splits into 9 Python modules, a small front end and a test suite. Game logic and lie handling are pure Python with no web code in them, so they can be unit tested on their own.

Architecture



Arrows point from the caller to what it uses. Only `local_client.py` reaches outside the game, and only to Ollama on the same laptop: nothing goes over the internet.

Modules

Module	Job	Covers
M1 `config.py`	All settings in one place: API key from env, timer length, offline/test mode defaults, random seed, file paths	NFR2, NFR4

Module	Job	Covers
M2 `models.py`	Data classes: Player, Team, Statement (text, is_true, source), Round, Game	FR1, FR2, FR4
M3 `game_engine.py`	Pure game logic: speaker order, coin-flip truth/lie pick, scoring, round counter, end of game, sudden death	FR3, FR7, FR13, FR15, FR16, FR22
M4 `ai_prompt.py`	The prompt and checks for AI lies: builds the prompt from players' stories and good example lies, reads the JSON reply, filters offensive output	FR5, FR17, NFR5
M5 `lie_bank.py`	Loads `lie_bank.json` (60+ lies), hands out unused entries, cycles the whole bank before repeating	FR6, FR20, FR21
M6 `lie_history.py`	Saves used lies to a local JSON file; checks exact and near-duplicate matches against the last 10 games; skipped in test mode	FR19, FR20, FR23, NFR6
M7 `lie_provider.py`	The one place the game asks for lies: picks online or offline, falls back to the bank on error, regenerates duplicates	FR5, FR6, FR19, FR21
M8 `app.py`	Flask routes and session: setup, story entry, privacy screen, round, guess, reveal, end screen, Play Again	FR1, FR2, FR4, FR8, FR12, FR14, FR16, NFR1
M9 `templates/`, `static/`	HTML pages, CSS, and `game.js` for the timer, Spacebar, Ready to Guess, reveal/hide, timer sound and test-mode banner	FR8 to FR11, FR18, FR23
M4b `local_client.py`	Talks to a free model running in Ollama on the laptop; same prompt as M4; clear error if Ollama is not running (CR-01)	FR24
M10 `tests/`	pytest unit and integration tests with a fake AI client	NFR8, all TCs

M2, M3, M5 and M6 never import Flask or call the internet. That is what lets most of the test cases run in under a second.

Folder structure

```
wilty/
├── app.py              M8
├── config.py           M1
├── wilty/
│   ├── models.py       M2
│   ├── game_engine.py  M3
│   ├── ai_prompt.py    M4
│   ├── local_client.py M4b
│   ├── lie_bank.py     M5
│   ├── lie_history.py  M6
│   └── lie_provider.py M7
├── data/
│   ├── lie_bank.json
│   └── lie_history.json   (created at runtime, git-ignored)
├── templates/          M9
├── static/
│   ├── css/style.css
│   └── js/game.js      (timer beep made in the browser, no sound file)
├── tests/              M10
│   ├── fake_ai.py
│   ├── test_engine.py
│   ├── test_lies.py
│   └── test_routes.py
├── requirements.txt
├── .env.example
└── .gitignore
```

Build order

Build from the inside out: logic first, web last. Each step ends with its tests passing, so nothing is built on untested code.

1. **M1 config + M2 models.** Done when the data classes can describe a full 3v3 game.
2. **M3 game engine.** Done when TC02, TC09, TC10, TC15, TC16 and TC20 pass. The whole game can now be played in a Python script with no website.
3. **M5 lie bank + M6 lie history.** Write the 60+ lies. Done when TC06, TC21, TC23 and TC24 pass.
4. **M4 prompt, M4b local client + M7 lie provider**, with `tests/fake_ai.py`. Done when TC04, TC05, TC07, TC08 and TC22 pass.
5. **M8 routes + M9 templates (no JavaScript yet).** Done when a full game can be clicked through with page reloads; TC01, TC03, TC19, TC25 and TC31 pass.
6. **M9 `game.js`.** Timer, Spacebar, Ready to Guess, reveal/hide, sound. Done when manual tests TC11 to TC14, TC17, TC18 and TC26 pass.
7. **Polish and real-world checks.** Local AI timing on the Mac (TC30), browsers (TC32), clean install (TC27), key safety (TC29), playtest (TC34).
8. **Deploy (later).** TC28.